

Лабораторная работа №1

Ansible: введение

1. Теоретические сведения

1.1. Ansible: назначение и решаемые проблемы

Ansible — инструмент автоматизации управления ИТ-инфраструктурой с открытым исходным кодом (Red Hat). Работает по принципу agentless: на управляемые узлы не нужно устанавливать дополнительное программное обеспечение — подключение выполняется по SSH или локально.

Ansible решает следующие задачи:

- Ручное администрирование — замена повторяющихся ручных операций на воспроизводимые сценарии.
- Дрейф конфигурации — гарантия того, что конфигурация сервера всегда соответствует описанию (идемпотентность).
- Масштабирование — управление сотнями хостов из одной точки одной командой.
- Документирование — конфигурация хранится в YAML-файлах в системе контроля версий (Infrastructure as Code).
- Воспроизводимость среды — одинаковые окружения dev / staging / production из одного кода.

1.2. Основные элементы Ansible

Роль — способ упаковать связанный набор задач, переменных, шаблонов и файлов в переиспользуемую единицу. Стандартная структура роли: tasks, handlers, vars, defaults, templates, files. Готовые роли публикуются на Ansible Galaxy.

Playbook — YAML-файл с одним или несколькими play. Каждый play задаёт: группу хостов (hosts), повышение привилегий (become), переменные (vars) и список задач (tasks). Каждая задача вызывает один модуль.

Инвентарь — файл (INI или YAML), в котором описываются управляемые хосты и параметры подключения к ним. Хосты объединяются в группы; для групп и хостов задаются переменные (ansible_user, ansible_connection и др.).

1.3. Основные модули ansible.builtin

Таблица 1 — Основные модули коллекции ansible.builtin

Модуль	Назначение
<code>ping</code>	Проверяет доступность хоста и Python-окружение. Не отправляет ICMP.
<code>command</code>	Выполняет команду без shell. Не поддерживает пайпы и перенаправление.
<code>shell</code>	Выполняет команду через /bin/sh. Поддерживает пайпы и переменные среды.
<code>copy</code>	Копирует файл на хост. Параметр content задаёт содержимое строкой.
<code>template</code>	Копирует Jinja2-шаблон с подстановкой переменных Ansible.
<code>file</code>	Управляет файлами и каталогами: создание, удаление, права (mode, owner).
<code>apt</code>	Управляет пакетами через APT (Debian/Ubuntu).
<code>service</code>	Управляет сервисами: запуск, остановка, автозагрузка.
<code>user</code>	Создаёт, изменяет и удаляет учётные записи пользователей.

<code>lineinfile</code>	Гарантирует наличие или отсутствие строки в файле конфигурации.
<code>debug</code>	Выводит сообщение или значение переменной в лог выполнения.
<code>set_fact</code>	Создаёт или переопределяет переменную хоста на лету.
<code>uri</code>	Выполняет HTTP/HTTPS-запросы; используется для проверки веб-сервисов.
<code>stat</code>	Возвращает информацию о файле: существование, тип, права, размер.

2. Цель работы

Получить практические навыки работы с Ansible: управление инвентарём (inventory), выполнение ad-hoc команд, написание и запуск playbook.

3. Подготовка к работе (15 мин)

Перед выполнением заданий необходимо:

1. Поднять виртуальную машину с операционной системой Ubuntu (рекомендуется Ubuntu 22.04 LTS). Все дальнейшие действия выполняются внутри этой ВМ.

2. Создать пользователя student — именно он указан в инвентарном файле (`ansible_user: student`). Пользователь должен иметь возможность выполнять команды с привилегиями суперпользователя (`sudo`) без запроса пароля, что необходимо для работы директивы `become: true` в playbook.

Выполнить от имени пользователя с правами администратора:

```
sudo adduser student
sudo usermod -aG sudo student
echo "student ALL=(ALL) NOPASSWD:ALL" | sudo tee
/etc/sudoers.d/student
```

Войти под пользователем student:

```
su - student
```

3. Установить Ansible. Выполнить следующие команды в терминале:

```
sudo apt update
sudo apt install software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install ansible
ansible --version
```

```
mkdir ~/ansible-lab && cd ~/ansible-lab
```

Задание 1. Инвентарный файл (10 мин)

Создать файл inventory.yml следующего содержания:

```
all:
  children:
    webservers:
      hosts:
        localhost:
          ansible_connection: local
          ansible_user: student
      vars:
        web_port: 80
```

Проверить корректность инвентарного файла командой:

```
ansible all -i inventory.yml --list-hosts
```

В отчёт: вывод команды --list-hosts.

Задание 2. Ad-hoc команды (15 мин)

Выполнить следующие ad-hoc команды Ansible. Для каждой команды в отчёте указать: назначение команды, используемый модуль, полученный вывод.

1. Проверка доступности хостов (модуль ping):

```
ansible all -i inventory.yml -m ping
```

2. Получение имени хоста (модуль command):

```
ansible all -i inventory.yml -m command -a "hostname"
```

3. Получение информации о дистрибутиве (модуль setup):

```
ansible all -i inventory.yml -m setup -a  
"filter=ansible_distribution"
```

4. Получение информации о ядре ОС (модуль setup):

```
ansible all -i inventory.yml -m setup -a  
"filter=ansible_kernel"
```

5. Создание файла с содержимым (модуль copy):

```
ansible all -i inventory.yml -m copy -a "content='Hello from  
Ansible' dest=/tmp/test.txt"
```

6. Чтение созданного файла (модуль command):

```
ansible all -i inventory.yml -m command -a "cat /tmp/test.txt"
```

В отчёт: для каждой команды — назначение, модуль, полученный вывод.

Задание 3. Playbook (25 мин)

3.1. Ознакомление с примером

Изучить структуру следующего playbook. Создать файл `example.yml` и запустить его:

```
---
- name: My first playbook
  hosts: webserver
  become: true
  vars:
    my_message: "Hello, Ansible!"
  tasks:
    - name: Show message
      debug:
        msg: "{{ my_message }}"
    - name: Create a file
      copy:
        content: "{{ my_message }}"
        dest: /tmp/hello.txt
    - name: Check file exists
      command: cat /tmp/hello.txt
      register: result
    - name: Print file content
      debug:
        msg: "{{ result.stdout }}"

ansible-playbook -i inventory.yml example.yml
```

3.2. Самостоятельное задание

Разработать playbook `site.yml`, который выполняет следующие задачи:

- 1) Устанавливает пакет `nginx`.
- 2) Запускает сервис `nginx` и добавляет его в автозагрузку.
- 3) Создаёт файл `/var/www/html/index.html`, содержащий имя студента и информацию о хосте (использовать переменные и факты Ansible).
- 4) Проверяет, что `nginx` отвечает на `http://localhost:80` (модуль `uri`).
- 5) Выводит HTTP-статус ответа через модуль `debug`.

Последовательность команд для проверки:

```
ansible-playbook -i inventory.yml site.yml --syntax-check
ansible-playbook -i inventory.yml site.yml
curl http://localhost
ansible-playbook -i inventory.yml site.yml    # второй запуск
(идемпотентность)
```

В отчёт: файл `site.yml`, вывод `PLAY RECAP` обоих запусков, вывод команды `curl http://localhost`.

Задание 4. Контрольные вопросы (5 мин)

Ответить письменно в отчёте на следующие вопросы:

1. Что означает значение `changed=0` при повторном запуске `playbook`?
2. Зачем в `playbook` используется директива `become: true`?
3. Чем модуль `command` отличается от модуля `shell`?
4. Что хранится в переменной `result` после выполнения задачи с модулем `uri`?

4. Содержание отчёта

Отчёт по лабораторной работе должен содержать:

1. Титульный лист.
2. Цель работы.
3. Файл `inventory.yml` и вывод команды `--list-hosts`.
4. Вывод всех `ad-hoc` команд (задание 2) с пояснениями.
5. Файл `site.yml`.
6. Вывод `PLAY RECAP` первого и второго запусков `playbook`.
7. Вывод команды `curl http://localhost`.
8. Ответы на контрольные вопросы (задание 4).
9. Выводы по работе.